
tracking-service (:4006)

Request flow, file by file

File-by-file request flow

Event-driven microservices · Go · Kafka · Redis

9 — tracking-service (:4006)

Job: the authority on donation status. Two completely separate entry points — this is the clearest example of a service that both consumes and serves.

```
== ENTRY A: Kafka (all six topics) ==  
  
index.js → events/consumer.js  startConsumer(topics, handlers)  
  | groupId: tracking-service    ▶ its OWN group → full stream  
  | fromBeginning: true  
  |  
  ▼  
events/consumer.js  handleMessage()  
  | JSON.parse  
  |   ▶ unparseable → log and COMMIT (a poison message would  
  |   otherwise block the whole partition forever)  
  |  
  | eventId + traceId from the BODY (header only as fallback)  
  |  
  | beginProcessing(eventId)    SETNX    config/redis.js  
  |   ▶ already processed → return, commit, done  
  |  
  | events/handlers.js  buildHandlers()[topic](event)  
  |   |  
  |   | domain/statusMachine.js  applyEvent(current, event)  
  |   |   ▶ PURE function, imports nothing  
  |   |   ▶ checks "already in this status" BEFORE legality,  
  |   |   so a routine redelivery is a silent no-op  
  |   |   ▶ terminal states accept nothing further  
  |   |  
  |   | models/DonationStatus.js  
  |   |   upsert + push onto the embedded timeline[]  
  |   |   + snapshot assignedAgentName / Phone  
  |   |   (no joins across services – so it is copied)  
  |   |  
  |   | on error → abandonProcessing(eventId) ▶ RELEASE the marker  
  |   | then rethrow ▶ offset NOT committed → redelivered
```

Entry B — HTTP

```
routes/tracking.routes.js      requireAuth
├ GET /stats/summary           ┌
├ GET /                       └─> controllers/tracking.controller.js
├ GET /:donationId             ┌
└ POST /:donationId/collected  └─> models/DonationStatus.js
                                requireRole('AGENT')
                                └─> events/producer.js publish donation.collected
                                    ▶ 202, not 200: the status changes only once
                                      the service consumes its OWN event back

routes/monitoring.routes.js    requireAuth + requireRole('ADMIN')
├ GET /stats
├ GET /donations
└ GET /donations/:donationId  ▶ full score breakdown per candidate
    ▶ EVERY route here is a GET. There is no write endpoint,
      deliberately – no assign button, no override.
```

Appendix — the shared skeleton

Repeated here so this page stands alone. Every Node service has these same files doing the same jobs.

Every Node service has these same files doing the same jobs. Once you know this, each service below is just "the skeleton, plus X".

```
src/
├─ index.js           ▶ bootstrap: connect dependencies, THEN listen
├─ app.js             ▶ build the middleware chain (no port binding)
├─ config/
│   ├─ env.js         ▶ read + validate env vars, fail fast at startup
│   ├─ db.js          ▶ mongoose connection
│   └─ redis.js       ▶ redis client (only where Redis is used)
├─ middleware/
│   ├─ requestContext.js ▶ x-trace-id in/out, attaches req.log
│   ├─ requireAuth.js  ▶ verify JWT → req.user; requireRole(...) gate
│   ├─ validate.js     ▶ schema check → 400 with field-level details
│   └─ errorHandler.js ▶ the ONE place every error is formatted
├─ routes/
│   ├─ *.routes.js    ▶ URL → middleware chain → controller
│   └─ *.schemas.js   ▶ the validation schemas
├─ controllers/       ▶ the actual work for one endpoint
├─ models/            ▶ mongoose schemas
└─ utils/
    ├─ jwt.js          ▶ signToken / verifyToken
    ├─ ApiError.js     ▶ typed errors (badRequest, notFound, forbidden...)
    └─ logger.js       ▶ structured JSON logging
```

The universal request pipeline



Why `index.js` and `app.js` are separate. `app.js` builds the Express app but never binds a port; `index.js` connects the database *first*, then listens. That split is what lets the tests drive the app in-process with `supertest` without binding a port or racing on one — and it means a service never accepts a request it cannot serve.

Why the error handler is last. Every failure, from any middleware or controller, funnels into one place. That is also where donation-service cleans up orphaned uploads — putting cleanup on the happy path missed every request that failed validation.